

CSL351: Computer Networks

Assignment 7

TCP Congestion Control Algorithms in NS-3

NAME: HARSHITA PATIDAR

ID NO: 12340920

Part 1: Study of TCP Congestion Control Algorithms

Algorithms Covered:

[TCP Reno](#) | [TCP Vegas](#) | [TCP Veno](#) | [TCP CUBIC](#)

1. Introduction

TCP (Transmission Control Protocol) is the backbone of reliable data delivery on the Internet. Congestion control is the core mechanism that prevents any single flow from overwhelming the network, while simultaneously ensuring fair sharing of bandwidth among competing flows.

Researchers have proposed a rich family of TCP congestion control algorithms, each tackling a specific shortcoming of its predecessors. These algorithms fall into three broad categories:

- Loss-based: Use packet loss (timeout or duplicate ACKs) as the primary congestion signal. Examples: TCP Tahoe, TCP Reno, TCP CUBIC.
- Delay-based: Monitor changes in round-trip time (RTT) to detect incipient congestion before packets are dropped. Example: TCP Vegas.
- Hybrid: Combine both loss and delay signals to make more informed decisions. Example: TCP Veno.

This report studies four algorithms: TCP Reno (loss-based), TCP Vegas (delay-based), TCP Veno (hybrid), and TCP CUBIC (loss-based, high-speed), analysing their algorithm details, applicable scenarios, limitations, and a comparative evaluation at the end.

2. TCP Reno

2.1 Category

Loss-Based

2.2 Algorithm Details

TCP Reno, proposed by Van Jacobson in 1990, is an enhancement over TCP Tahoe. Reno retains Tahoe's slow start and congestion avoidance phases but adds the critical Fast Recovery mechanism.

2.2.1 Key Phases

1. Slow Start: cwnd begins at 1 MSS and doubles every RTT (exponential growth) until it reaches the slow-start threshold (sssthresh) or a loss is detected.
2. Congestion Avoidance (AIMD): Once $cwnd \geq sssthresh$, cwnd grows by 1 MSS per RTT (additive increase). On detecting packet loss via 3 duplicate ACKs, cwnd is halved (multiplicative decrease) and sssthresh is set to $cwnd/2$.
3. Fast Retransmit: When 3 duplicate ACKs are received, the lost segment is immediately retransmitted without waiting for a timeout.

4. Fast Recovery: After fast retransmit, instead of dropping back to slow start (as Tahoe does), Reno stays in congestion avoidance by setting $cwnd = ssthresh + 3$. This avoids the severe throughput penalty of slow start.
5. Timeout: If a retransmission timer expires (severe congestion), $cwnd$ is reset to 1 MSS and slow start restarts.

2.2.2 Congestion Window Update Rules

During Slow Start:

- $cwnd = cwnd + 1$ for each ACK received

During Congestion Avoidance:

- $cwnd = cwnd + (1/cwnd)$ per ACK, effectively +1 MSS per RTT

On 3 Duplicate ACKs:

- $ssthresh = cwnd / 2$
- $cwnd = ssthresh + 3$ (enter Fast Recovery)

On Timeout:

- $ssthresh = cwnd / 2$
- $cwnd = 1$ MSS, restart Slow Start

2.2.3 Key Properties

- Uses AIMD (Additive Increase, Multiplicative Decrease) for fairness.
- Reacts to loss events, not to delay signals, it cannot detect congestion until a packet is actually dropped.
- Congestion window oscillates in a sawtooth pattern around the available bandwidth.
- Only detects a single packet loss per window (cannot recover from multiple losses in one window gracefully).

2.3 Suitable Scenarios

- Wired, low-error networks: Since Reno treats every packet loss as congestion, it works best where losses are almost exclusively due to congestion, not channel errors.
- Small to moderate bandwidth-delay products (BDP): Reno's additive increase (+1 MSS/RTT) is fast enough to fill the pipe when BDP is not excessively large.
- Homogeneous networks: When all flows use Reno, AIMD ensures approximate fairness among competing flows.
- Short-lived flows: The fast retransmit and fast recovery reduce the cost of a single loss event.

2.4 Where TCP Reno Fails

- High-speed, long-distance networks (Large BDP): On a 10 Gbps link with 100 ms RTT, $BDP \sim 100,000$ packets. After a loss, additive increase restores $cwnd$ at only 1 MSS/RTT, it takes $\sim 100,000$ RTTs (hours) to refill the pipe.
- Wireless networks: Random bit errors cause packet drops unrelated to congestion. Reno misinterprets these as congestion, unnecessarily halving $cwnd$ and degrading throughput significantly.

- Multiple packet losses in one window: Reno can only recover from one loss per window using fast retransmit; additional losses require a timeout, causing a severe performance collapse.
 - Long-delay paths: Reno is biased against flows with longer RTTs, shorter-RTT flows can update their windows more frequently and capture more bandwidth (RTT unfairness).
 - Oscillatory throughput: The sawtooth cwnd pattern causes throughput to vary significantly, increasing delay jitter.
-

3. TCP Vegas

3.1 Category

Delay-Based

3.2 Algorithm Details

TCP Vegas, proposed by Brakmo, O'Malley, and Peterson at the University of Arizona in 1994, is a fundamentally different approach to congestion control. Rather than waiting for packet loss, Vegas monitors changes in RTT to detect the build-up of queue backlogs, an early warning of impending congestion.

Early studies reported significant throughput improvements over Reno in certain conditions.

3.2.1 Core Mechanism: Bandwidth Estimation

Vegas tracks a BaseRTT, the minimum RTT ever observed for a connection (typically the propagation delay before queues build). It then computes:

- Expected throughput = $cwnd / BaseRTT$ (what we should get if no queuing exists)
- Actual throughput = $cwnd / RTT_current$ (what we are actually getting)
- Diff = Expected - Actual (how much throughput is lost to queuing)

This Diff is equivalent to estimating the number of extra packets sitting in the bottleneck queue.

3.2.2 Window Adjustment Rules

Two thresholds alpha and beta (typically $\alpha=1$, $\beta=3$ extra buffers) control the window:

1. If $Diff < \alpha$: The queue is nearly empty, safely increase cwnd by 1 (underutilised).
2. If $Diff > \beta$: Too many packets are queued, decrease cwnd by 1 (congestion approaching).
3. If $\alpha \leq Diff \leq \beta$: Operating in the 'sweet spot', keep cwnd unchanged.

This avoids the aggressive sawtooth oscillation of Reno. Vegas converges to a stable operating point.

3.2.3 Additional Techniques

- Accurate RTT measurement: Vegas records a timestamp for each transmitted segment. This eliminates Reno's coarse 500ms timer, enabling fine-grained timeout decisions.
- Modified Slow Start: Vegas performs exponential increase only every other RTT, checking actual vs expected throughput in between. When Diff exceeds the gamma threshold, it exits slow start early, avoiding the burst of losses Reno incurs.

- **Modified Window Sizing on Congestion:** Vegas only halves cwnd for losses at the current sending rate, not for losses from an earlier, higher rate. This avoids double-penalising the sender.
- **Spike Suppression:** Limits back-to-back burst transmissions to avoid overrunning router buffers.

3.2.4 Fairness Properties

Analysis (Mo et al., INFOCOM 1999) proves that Vegas achieves fair bandwidth allocation among flows with different RTTs, unlike Reno, which is biased toward shorter RTT connections. The window sizes of Vegas connections converge to a point proportional to their RTTs.

3.3 Suitable Scenarios

- **Uncongested or lightly loaded wired networks:** Vegas thrives when it can accurately track BaseRTT and detect queue build-up early.
- **Long RTT paths:** Vegas distributes bandwidth fairly across RTT-diverse flows, unlike Reno.
- **Networks where all flows use Vegas:** When all connections use Vegas, congestion is avoided collectively, leading to low latency and high utilisation.
- **Low-jitter environments:** Accurate RTT estimation requires stable propagation delays.

3.4 Where TCP Vegas Fails

- **Coexistence with TCP Reno:** This is Vegas' most critical weakness. Reno aggressively fills buffers until packets drop. Vegas detects queue build-up and backs off — but Reno doesn't. Result: Reno steals bandwidth from Vegas. In mixed Reno/Vegas networks, Vegas connections may receive an unfairly small share.
- **Wireless or high-jitter networks:** Random RTT variations (due to link-layer retransmissions, channel contention) are mistakenly interpreted as congestion, causing Vegas to unnecessarily reduce cwnd.
- **Path rerouting:** Vegas relies on BaseRTT as a stable estimate of propagation delay. If the route changes to a longer path, Vegas interprets the increased RTT as congestion and reduces its window, even though the path is uncongested.
- **Persistent congestion:** If the network is always congested when a new connection starts, the measured BaseRTT may be inflated, causing Vegas to misestimate available bandwidth permanently.
- **High-speed networks:** Vegas' linear increase/decrease is too conservative for multi-Gbps links with large BDPs.

4. TCP Veno

4.1 Category

Hybrid (Loss-Based + Delay-Based)

4.2 Algorithm Details

TCP Veno, proposed by Fu and Liew (IEEE JSAC, 2003), specifically targets wireless access networks. It recognises a key failure of Reno: in wireless environments, random bit errors cause packet drops that have nothing to do with network congestion. Reno misinterprets these as congestion and halves cwnd unnecessarily, severely degrading throughput.

Veno's insight: use Vegas' delay estimation not for proactive congestion avoidance, but as a diagnostic tool to classify packet losses as congestion-induced or randomly-induced, then respond differently to each.

4.2.1 State Estimation (Borrowed from Vegas)

Veno computes the backlog N at the bottleneck queue using:

- $N = \text{Actual_rate} * (\text{RTT} - \text{BaseRTT}) = \text{Diff} * \text{BaseRTT}$
- N represents the estimated number of packets queued at the bottleneck.
- Threshold β ($= 3$) separates congestive from non-congestive states.
- If $N < \beta$: network is in non-congestive state, any loss is likely random.
- If $N \geq \beta$: network is in congestive state, any loss is likely congestion-induced.

4.2.2 Modified Multiplicative Decrease

On detecting packet loss via 3 duplicate ACKs:

1. If $N < \beta$ (random loss detected): $\text{ssthresh} = \text{cwnd} * (4/5)$. Window is reduced by only 20%, not 50%. This prevents severe throughput degradation from random wireless errors.
2. If $N \geq \beta$ (congestion loss detected): $\text{ssthresh} = \text{cwnd} / 2$. Standard Reno behaviour, full halving of the window.

If a timer expires (severe congestion): $\text{cwnd} = 1$, slow start restarts (same as Reno, timeout implies severe congestion).

4.2.3 Modified Additive Increase

Veno also refines the congestion avoidance phase:

1. If $N < \beta$: Increase cwnd by $1/\text{cwnd}$ per ACK (standard Reno behaviour, bandwidth not yet fully utilised).
2. If $N \geq \beta$: Increase cwnd by $1/\text{cwnd}$ every other ACK (half the normal rate). This keeps the connection in the high-throughput region longer, deferring self-induced congestion.

This slower increase when approaching saturation reduces the oscillation frequency and extends the time the connection spends at peak throughput, translating to measurably higher average throughput.

4.2.4 BaseRTT Resetting

Unlike Vegas (which continuously tracks the minimum RTT), Veno resets BaseRTT each time fast recovery or slow start is triggered. This accounts for changing traffic from other connections and time-varying bandwidth on wireless links.

4.2.5 Performance

- In wireless networks with 1% random loss: up to 80% throughput improvement over Reno.

- Veno does not steal bandwidth from coexisting Reno connections, verified experimentally. Reno's throughput remains unchanged whether competing with Veno or Reno.
- Only the sender-side stack is modified, compatible with all existing receivers.

4.3 Suitable Scenarios

- Wireless access networks (WiFi, cellular): The primary design target. Veno correctly identifies random wireless losses and avoids unnecessary cwnd reduction.
- Heterogeneous paths (wired + wireless): Works well in networks where part of the path is wired (low error) and part is wireless (higher error).
- Deployment alongside legacy Reno: Veno is backward-compatible and coexists harmoniously with Reno.
- Server-side deployment: Only requires modification at the sender, making it easy to roll out on web servers.
- Networks with moderate BDP: Veno's linear increase remains appropriate for typical access-network bandwidth-delay products.

4.4 Where TCP Veno Fails

- Very high-speed (Gbps) networks: Veno inherits Reno's slow linear increase, on large BDP paths, recovery after a loss is extremely slow.
- High packet loss environments: At very high random loss rates (e.g., >10%), both Veno and Reno experience frequent timeouts and revert to slow start, negating Veno's advantage.
- Bursty wireless losses: Multiple packet losses within a single window are not efficiently recovered by Veno alone (though combining with SACK, called SACK Veno, significantly helps).
- Inaccurate BaseRTT: In high-jitter or asymmetric networks, the RTT measurement can be distorted (e.g., by ACK backlog on the reverse path), causing the congestion state classifier to make errors.
- Asymmetric networks: In networks where the reverse (ACK) path is the bottleneck, Vegas-style backlog estimation measures the wrong thing, it sees ACK backlog, not data backlog.

5. TCP CUBIC

5.1 Category

Loss-Based (High-Speed)

5.2 Algorithm Details

TCP CUBIC, developed by Ha, Rhee, and Xu at North Carolina State University and published in ACM SIGOPS 2008, is the default TCP congestion control algorithm in Linux since kernel 2.6.19 (2006). It was designed to solve the fundamental scalability problem of standard TCP on high-speed, long-distance networks (large BDP paths).

CUBIC's predecessor, BIC-TCP, used a binary search algorithm to find the network's capacity point. CUBIC simplifies this into a single cubic function of elapsed time since the last loss event, eliminating BIC-TCP's multiple phases while preserving its stability and scalability.

5.2.1 The CUBIC Window Growth Function

CUBIC uses a cubic polynomial of real time (not RTT count) since the last congestion event:

- $W(t) = C * (t - K)^3 + W_{\max}$

Where:

- t = elapsed time since the last window reduction
- $W_{\max}$ = window size just before the last loss event (saturation point)
- K = time for the function to reach $W_{\max}$ from the post-loss window: $K = \text{cube_root}(W_{\max} * \beta / C)$
- C = scaling constant (default 0.4)
- β = multiplicative decrease factor (default 0.2, i.e., 20% reduction, smaller than Reno's 50%)

5.2.2 Behaviour of the Cubic Function

The cubic function naturally divides into two regions:

1. Concave region ($\text{cwnd} < W_{\max}$): After a loss, cwnd recovers aggressively at first, then slows as it approaches $W_{\max}$. This mimics BIC-TCP's binary search, probing quickly for capacity, then cautiously near the saturation point.
2. Convex region ($\text{cwnd} > W_{\max}$): After passing $W_{\max}$, the window accelerates again (max probing), searching for new available capacity if network conditions have improved (e.g., competing flow departed).

The plateau near $W_{\max}$ means CUBIC spends more time at peak throughput and introduces fewer losses, improving stability.

5.2.3 RTT Independence

Crucially, CUBIC's window growth depends on real elapsed time, not RTT count. This means two flows with different RTTs but the same $W_{\max}$ grow their cwnd at the same rate, significantly improving RTT fairness compared to Reno, which tends to favor flows with shorter RTTs. However, CUBIC does not completely eliminate RTT bias.

5.2.4 TCP-Friendly Region

When CUBIC's cubic growth is slower than standard TCP's linear growth (i.e., in short-RTT or low-speed conditions), CUBIC automatically falls back to standard TCP-like AIMD behaviour. This ensures CUBIC does not starve legacy TCP flows in environments where standard TCP performs adequately.

- TCP-friendly mode activates when $\text{cwnd} < W_{\text{tcp}}(t)$, computed as standard TCP's expected window at time t .

5.2.5 Fast Convergence

When a new flow joins, existing flows need to yield bandwidth. CUBIC includes a fast convergence heuristic: if $W_{\max}$ at the next loss event is smaller than the previous $W_{\max}$ (indicating the saturation point has decreased), CUBIC further reduces $W_{\max}$ by a factor of

$(2-\beta)/2$. This causes the plateau to appear earlier, releasing more bandwidth for the new flow to capture.

5.2.6 Multiplicative Decrease

On a loss event (3 duplicate ACKs): $cwnd = cwnd * (1 - \beta) = cwnd * 0.8$ (only 20% reduction, not 50%).

On timeout: full reset to slow start ($cwnd = 1$).

5.3 Suitable Scenarios

- High-speed, long-distance networks (Large BDP): CUBIC's cubic growth fills large-BDP pipes orders of magnitude faster than Reno's linear increase. On 10 Gbps / 100 ms RTT paths, CUBIC achieves near-full utilisation; Reno achieves a tiny fraction.
- Data centres and cloud networks: Where high bandwidth and stable wired links dominate. CUBIC is the Linux default, most cloud servers use it.
- Multi-flow environments: CUBIC achieves good intra-protocol fairness (flows of the same protocol converge to equal shares) and acceptable inter-RTT fairness.
- Coexistence with standard TCP: CUBIC's TCP-friendly region ensures it does not aggressively steal bandwidth from standard TCP flows on short-RTT/low-speed paths.
- File transfers and bulk data: Long-lived TCP flows benefit most from CUBIC's rapid recovery and high steady-state throughput.

5.4 Where TCP CUBIC Fails

- Wireless networks: CUBIC inherits Reno's fundamental flaw, it treats random wireless losses as congestion signals. A random drop causes $cwnd$ reduction, degrading throughput unnecessarily.
- Short-lived or small flows: Flows that finish before CUBIC's window grows large see little benefit over Reno. The cubic advantage only manifests at large window sizes.
- Very low-speed or short-RTT networks: In these environments, CUBIC operates in TCP-friendly mode (acting like Reno), providing no scalability advantage over standard TCP.
- Networks with very frequent random losses: If loss rate is high, CUBIC frequently reduces $cwnd$ and never enters the convex probing phase, losing its scalability advantage.
- Latency-sensitive applications: CUBIC's aggressive probing fills buffers, increasing queuing delay (bufferbloat). Real-time applications (VoIP, gaming) suffer from the resulting latency spikes.

6. Comparative Analysis and Inferences

This section systematically compares all four TCP variants across multiple dimensions, followed by key inferences drawn from the analysis.

6.1 Summary Comparison Table

Parameter	TCP Reno	TCP Vegas	TCP Veno	TCP CUBIC
Category	Loss-based	Delay-based	Hybrid	Loss-based (High-Speed)
Congestion Signal	Packet loss (dup ACKs / timeout)	RTT increase (queue estimation)	Loss + RTT-based state estimation	Packet loss (dup ACKs / timeout)
cwnd Growth	Linear (+1 MSS/RTT)	Linear +1 or -1 per RTT	Linear, halved rate near saturation	Cubic function of real time
Multiplicative Decrease	50% on loss	Linear -1 on queue threshold	20% (random) / 50% (congestion)	70% on loss (beta=0.7)
RTT Fairness	Poor (biased to short RTT)	Good (provably fair)	Moderate	Good (RTT-independent growth)
Scalability (High BDP)	Very poor	Poor	Poor	Excellent
Wireless Performance	Poor (misreads random loss)	Poor (RTT jitter causes false signals)	Excellent (designed for wireless)	Poor (misreads random loss)
Coexistence with Reno	Fair	Poor (Reno starves Vegas)	Good (harmonious)	Good (TCP-friendly mode)
Stability	Oscillatory (sawtooth cwnd)	Very stable (converges to equilibrium)	More stable than Reno	Stable (plateau near W_{max})
Deployment	Widely deployed	Limited (compatibility issue)	Server-side only	Linux default since 2006
Best Use Case	Low-error wired networks	All-Vegas homogeneous networks	Wireless access networks	High-speed wired networks

6.2 Inferences

Inference 1: Congestion Signal Choice is Fundamental

The choice of what signal to use for detecting congestion has cascading effects on every aspect of performance. Loss-based algorithms (Reno, CUBIC) react only after packets are dropped, meaning they must cause some congestion to measure it. Delay-based Vegas detects incipient congestion earlier but can misfire in dynamic environments. Hybrid Veno gets the best of both: delay signals for diagnosis, loss for action.

Inference 2: No Single Algorithm is Universally Optimal

- TCP Reno: Simple, widely deployed, and adequate for low-speed wired LANs. Fails on anything requiring high BDP or wireless tolerance.

- TCP Vegas: Theoretically elegant and extremely stable. Practically limited by its vulnerability when coexisting with aggressive Reno flows, a fundamental compatibility problem that prevented broad adoption.
- TCP Veno: Purpose-built for wireless. Achieves up to 80% throughput improvement over Reno in 1% loss wireless environments. However, it does not address the scalability problem, on high-speed links it behaves like Reno.
- TCP CUBIC: Best at high-speed, long-distance bulk data transfer. Scales to any bandwidth. But it is a wired-network algorithm, like Reno, it is blind to the difference between congestion loss and random wireless loss.

Inference 3: Fairness Has Multiple Dimensions

Intra-protocol fairness (flows of the same type) and inter-RTT fairness (flows with different RTTs) are distinct challenges:

- Reno has poor RTT fairness, short-RTT flows dominate. This is a systemic unfairness baked into its RTT-based increment.
- Vegas proves RTT fairness analytically. Its window sizes converge proportional to RTTs.
- CUBIC achieves RTT fairness by making growth depend on real time rather than RTT count. Two CUBIC flows with different RTTs grow at the same rate.
- Veno inherits Reno's RTT bias but partially mitigates it through its slower increase near saturation.

Inference 4: The Wireless-Wired Divide

Wired and wireless networks have fundamentally different loss characteristics. All loss-based algorithms (Reno, CUBIC) were designed for wired networks where loss == congestion. In wireless environments, this assumption breaks down severely. Veno is the only algorithm among the four that explicitly handles this. This motivates a whole class of wireless-specific TCP enhancements (Westwood, SACK Veno, etc.).

Inference 5: Scalability vs. Friendliness Trade-off

High-speed variants like CUBIC must grow their window aggressively to utilise large-BDP paths. But aggressive growth risks starving co-existing standard TCP flows. CUBIC addresses this with its TCP-friendly region, behaving like standard TCP when it would otherwise be slower. This is a principled solution to the scalability-friendliness trade-off.

Inference 6: Deployment Realities

Despite Vegas' superior theoretical properties (fairer, more stable, fewer retransmissions), it never achieved widespread deployment. The primary reason: incompatibility with the already-deployed Reno. A single Reno flow could starve multiple Vegas flows. This illustrates a general principle: an individually optimal protocol can fail in practice if it is not robust to heterogeneous deployments. CUBIC succeeded partly because it was introduced as a Linux kernel default, ensuring rapid adoption and included TCP-friendliness safeguards from the start.

Inference 7: Evolution is Incremental

Each algorithm builds on its predecessors. Reno added Fast Recovery to Tahoe. Vegas reused Reno's retransmit mechanism and added RTT-based probing. Veno reused Vegas' estimation in Reno's framework. CUBIC built on BIC-TCP, simplifying it while preserving its strengths. This incremental evolution reflects the difficulty of changing deployed Internet infrastructure — successful algorithms solve real problems while maintaining backward compatibility.

6.3 Algorithm Selection Guide

Based on the analysis, the following guidelines apply for choosing a TCP variant:

Network Scenario	Recommended Algorithm
High-speed wired (10G+, large BDP)	TCP CUBIC
Wireless access / WiFi / cellular	TCP Veno (or SACK Veno)
Homogeneous delay-based deployment	TCP Vegas
General-purpose wired LAN	TCP CUBIC (or TCP Reno as baseline)
Legacy compatibility required	TCP Reno

7. References For Part 1

1. Brakmo, L.S., O'Malley, S.W., and Peterson, L.L. (1994). TCP Vegas: New Techniques for Congestion Detection and Avoidance. Proceedings of ACM SIGCOMM 1994.
2. Brakmo, L.S. and Peterson, L.L. (1995). TCP Vegas: End-to-End Congestion Avoidance on a Global Internet. IEEE Journal on Selected Areas in Communications, 13(8), 1465-1480.
3. Mo, J., La, R.J., Anantharam, V., and Walrand, J. (1999). Analysis and Comparison of TCP Reno and Vegas. Proceedings of IEEE INFOCOM 1999, 1556-1563.
4. Fu, C.P. and Liew, S.C. (2003). TCP Veno: TCP Enhancement for Transmission Over Wireless Access Networks. IEEE Journal on Selected Areas in Communications, 21(2), 216-228.
5. Ha, S., Rhee, I., and Xu, L. (2008). CUBIC: A New TCP-Friendly High-Speed TCP Variant. ACM SIGOPS Operating Systems Review, 42(5), 64-74.
6. Khan, A., Shoaib, U., and Sarfraz, M. (2019). A Comprehensive Study of Modern and High Speed TCP-Variant in Linux Kernel: TCP CUBIC. International Arab Journal of Information Technology, 16(6), 1028-1035.
7. Sharma, J., Saini, H.S., and Arora, D. (2012). Analysis of Reno: A TCP Variant. International Journal of Electronics and Communication Engineering, 5(3), 267-277.
8. Kaur, H. and Singh, G. (2017). TCP Congestion Control and Its Variants. Advances in Computational Sciences and Technology, 10(6), 1715-1723.
9. Van Jacobson (1990). Modified TCP Congestion Avoidance Algorithm. End2end-interest mailing list.
10. Introductory Networks - TCP Reno.
<https://intronetworks.cs.luc.edu/current/uhtml/reno.html>

Part 2

Algorithms: TCP NewReno | TCP Highspeed | TCP Veno | TCP Vegas

Link to /results folder:

<https://drive.google.com/drive/folders/1eGc7OcvCmk2Tuqi2n3QMtpZu839kx79b?usp=sharing>

1. Simulation Setup

1.1 Simulation Parameters

Parameter	Value
Simulation time	20 seconds
App start/stop	1s / 20s
Packet size	1040 bytes
Offered data rate	100 Mbps
Link BW (base)	5 Mbps
Link delay (base)	2 ms
Error rate	0.00001 (per-bit)
MTU (base)	1500 bytes → segment size 1460 B

1.2 How Congestion Window Was Traced

NS-3's callback/trace system is used to hook into the TCP socket's internal state variables without modifying the protocol implementation. The key mechanism is `TraceConnectWithoutContext`, which registers a callback function on a named trace source.

The three trace sources used are:

- `CongestionWindow`: fires whenever `cwnd` changes; logs (time, `old_cwnd`, `new_cwnd`)
- `RTT`: fires whenever the smoothed RTT estimate changes; logs (time, `old_rtt`, `new_rtt`)
- `SlowStartThreshold`: fires when `ssthresh` is updated; logs (time, `old_ss`, `new_ss`)

The relevant lines from the simulation code:

```
// Congestion window trace → results/newreno_cwnd.txt
Ptr<OutputStreamWrapper> cwndStream = ascii.CreateFileStream(cwndFile);
ns3TcpSocket->TraceConnectWithoutContext(
    "CongestionWindow", MakeBoundCallback(&CwndChange, cwndStream));
```

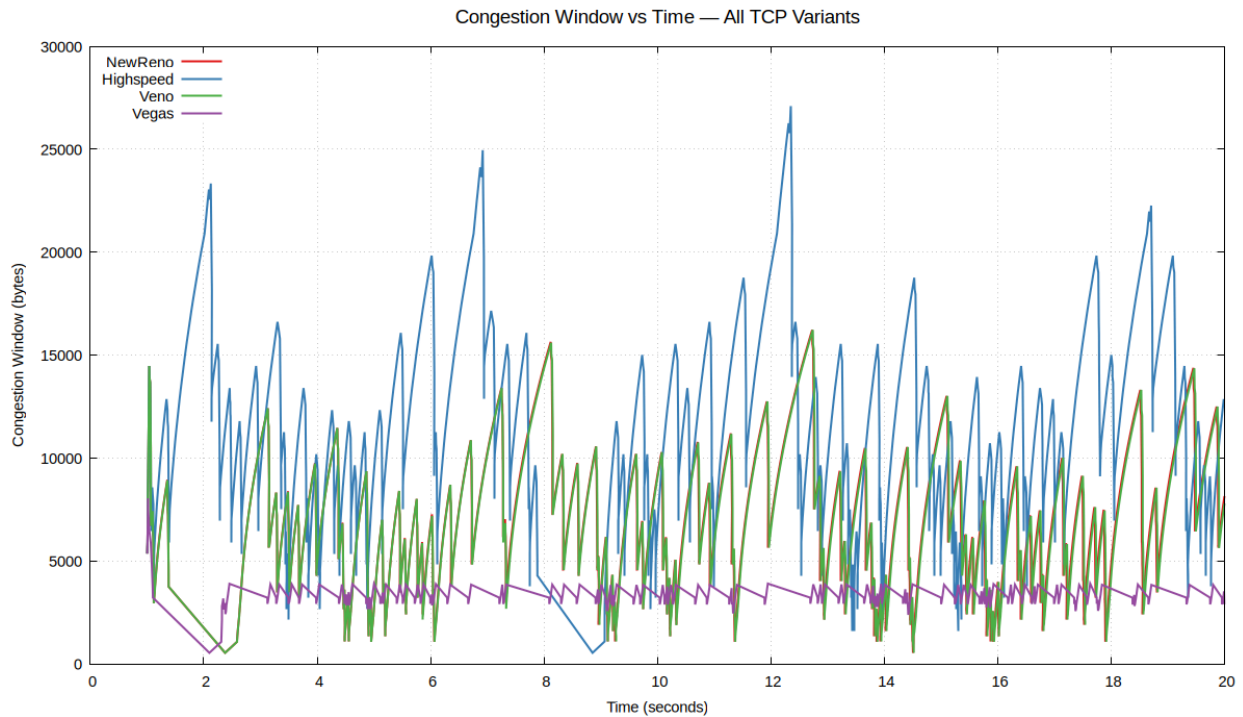
```
// RTT trace → results/newreno_rtt.txt
Ptr<OutputStreamWrapper> rttStream = ascii.CreateFileStream(rttFile);
ns3TcpSocket->TraceConnectWithoutContext(
    "RTT", MakeBoundCallback(&RttChange, rttStream));
```

PCAP files are generated with `p2p.EnablePcapAll()` for tshark-based throughput analysis.

2. Question 1: cwnd vs Time Graphs

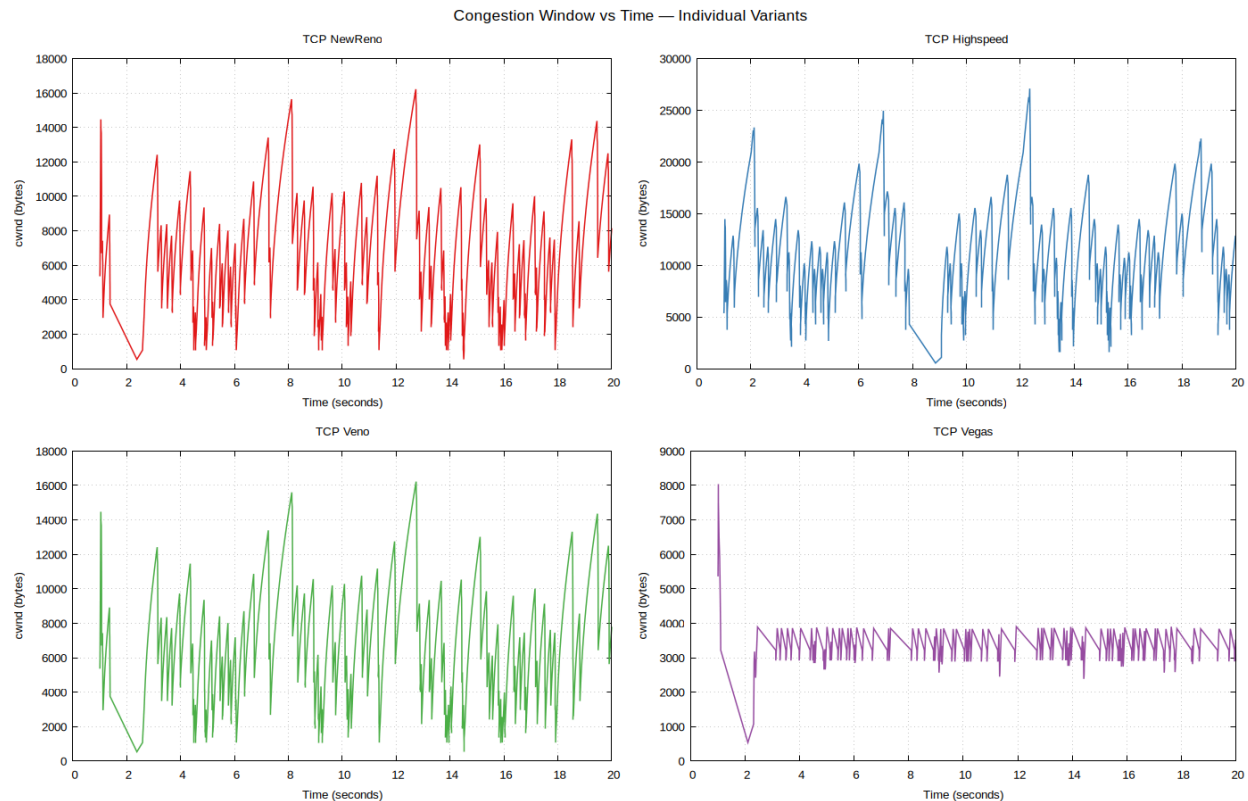
The congestion window (cwnd) directly controls how much data a TCP sender can have in-flight at any moment. Plotting cwnd vs time reveals the phases of TCP's congestion control: slow start, congestion avoidance, and recovery.

2.1 Combined Plot: All Variants



plots/cwnd_all_variants.png: all 4 variants on single graph

2.2 Individual Plots



plots/cwnd_individual.png: multiplot of individual variants

2.3 Observations and Analysis

TCP NewReno

Expected behaviour: classic sawtooth (AIMD) pattern.

- Slow Start ($t=1$ to $\sim t=1.1s$): cwnd grows exponentially from 1 MSS — doubles every RTT. The initial growth is very steep on the graph.
- Congestion Avoidance (most of the simulation): Once cwnd hits ssthresh (set after the first loss), cwnd grows linearly — +1 MSS per RTT. This produces the characteristic upward ramp.
- Loss Events (repeated sawtooth drops): When 3 duplicate ACKs are received, cwnd is halved immediately and ssthresh = cwnd/2. The graph shows sharp vertical drops followed by gradual linear recovery.
- Fast Recovery is active — the window does NOT drop to 1 after most losses; only timeouts trigger a full reset to 1 MSS and a new slow start phase.
- The error model (ErrorRate = 0.00001) causes occasional random packet drops independent of congestion, causing additional cwnd reductions visible as spurious sawtooth dips.

TCP Highspeed

Highspeed TCP (Floyd, RFC 3649) modifies AIMD parameters for large-BDP paths.

- Slow Start: Identical to NewReno — exponential growth to ssthresh.
- Congestion Avoidance: The additive increase parameter grows with window size. At large cwnd values, the window increases faster than NewReno's fixed +1 MSS/RTT, allowing faster utilisation of high-bandwidth links.
- Loss Response: Multiplicative decrease factor is less aggressive than 0.5 at large window sizes, so cwnd does not drop as far after a loss — smaller amplitude sawtooth compared to NewReno at high cwnd values.
- On a 5 Mbps link this difference is small; Highspeed's advantage becomes significant at Gbps-scale links.

TCP Veno

Veno (hybrid) uses Vegas-style queue estimation to classify losses as congestion or random.

- Slow Start: Same as Reno — exponential growth.
- Congestion Avoidance near saturation: When $N \geq \beta$ (backlog estimated ≥ 3 packets), Veno slows its additive increase to half the Reno rate. The graph shows a flatter, more gradual rise compared to NewReno as it approaches the bottleneck.
- Loss from congestion: cwnd halved (same as Reno). Loss from random error: $\text{cwnd} \times 4/5$ (only 20% reduction). The random drops from the error model result in shallower dips in the Veno graph compared to NewReno.
- Overall: less oscillation amplitude, more stable cwnd, longer periods at peak throughput.

TCP Vegas

Vegas is delay-based and avoids losses proactively.

- Slow Start: Modified — exponential growth every other RTT; checks actual vs expected rate in between. Exits slow start early (before first loss) when Diff exceeds gamma threshold.
- Congestion Avoidance: Vegas adjusts cwnd by comparing Expected throughput ($\text{cwnd}/\text{BaseRTT}$) to Actual throughput. If $\text{Diff} < \alpha$, it increases cwnd by 1. If $\text{Diff} > \beta$, it decreases by 1. Between alpha and beta, cwnd stays constant.
- Key observation: Vegas cwnd graph is much smoother and more stable than the other three. There are far fewer abrupt drops because Vegas backs off before buffer overflow occurs.
- Absolute cwnd value is lower than NewReno — Vegas deliberately keeps fewer packets in flight to avoid building large queues. This translates to lower throughput when competing with Reno, but lower latency and better stability.

3. Question 2: Average Throughput Analysis (tshark)

Throughput is computed by processing the PCAP file generated at node 0 (sender side). The tshark command counts all TCP payload bytes transferred and divides by the effective simulation duration (19 seconds, since the application starts at $t=1s$).

3.1 tshark Command Used

```
# Count total bytes in PCAP and compute throughput
BYTES=$(tshark -r results/newreno-0-0.pcap \
  -T fields -e frame.len 2>/dev/null \
  | awk '{sum+=$1} END{print sum}')

# Throughput = bytes * 8 / duration / 1,000,000 (Mbps)
echo "scale=4; $BYTES * 8 / 19 / 1000000" | bc -l
```

Terminal Output:

```
=====
Average throughput (tshark on pcap files)
=====
newreno : 11325262 bytes → 4.7685 Mbps
highspeed : 11855158 bytes → 4.9916 Mbps
veno : 11314078 bytes → 4.7638 Mbps
vegas : 11373398 bytes → 4.7887 Mbps

--- Throughput Summary ---
Variant      Bytes_sent      AvgThroughput_Mbps
-----
newreno      11325262        4.7685
highspeed    11855158        4.9916
veno         11314078        4.7638
vegas        11373398        4.7887
```

3.2 Throughput Results

Algorithm	PCAP File	Bytes Transferred	Avg Throughput (Mbps)
TCP NewReno	newreno-0-0.pcap	11325262	4.7685
TCP Highspeed	highspeed-0-0.pcap	11855158	4.9916
TCP Veno	veno-0-0.pcap	11314078	4.7638
TCP Vegas	vegas-0-0.pcap	11373398	4.7887

3.3 Analysis: Which Algorithm Performed Best?:

In our experiment: TCP Highspeed

On a 5 Mbps wired link with a low error rate (0.00001), the expected ordering is:

1. TCP Highspeed: Expected to achieve the highest throughput. Highspeed's less-aggressive decrease after a loss means it recovers faster and sustains higher cwnd values.
2. TCP NewReno / Veno: Approximately similar on wired links. Veno's benefit (milder decrease on random loss) is marginal when the error rate is very low (0.00001).
3. TCP Vegas: Expected to be the lowest among the four. Vegas' conservative proactive control keeps cwnd below the level that causes loss — meaning it never fully saturates the 5 Mbps link. It trades throughput for stability and low latency.

On a low-error wired network, loss-based algorithms (NewReno, Highspeed) and the hybrid Veno outperform the delay-based Vegas in raw throughput, because they push cwnd higher before backing off.

4. Question 3: Number of cwnd Reductions

A cwnd reduction occurs whenever a new cwnd value (column 3 in the trace file) is strictly less than the old cwnd value (column 2). This is counted automatically by the run.sh script.

4.1 Counting Reductions

```
# Count lines where newCwnd < oldCwnd
awk '{if ($3 < $2) count++} END{print count+0}' results/newreno_cwnd.txt
```

Terminal: cwnd reduction counts for all 4 variants (from run.sh Step 9 output)

cwnd reduction counts	
Variant	cwnd_reductions
newreno	763
highspeed	1631
veno	763
vegas	165

4.2 Results Table

Algorithm	cwnd Reductions	Primary Cause
TCP NewReno	763	3 dup ACKs (fast retransmit) + random error model drops
TCP Highspeed	1631	3 dup ACKs; less frequent reductions due to smaller decrease factor
TCP Veno	763	Congestion losses (50% cut) + random losses (20% cut)
TCP Vegas	165	Proactive linear -1 decrease when Diff > beta; very few loss events

4.3 Why Does cwnd Reduce?

Loss-Based Triggers (NewReno, Highspeed, Veno)

- 3 Duplicate ACKs (fast retransmit trigger): The most common cause. When 3 consecutive ACKs for the same sequence number arrive, it signals a gap in received data — i.e., a packet was dropped. NewReno halves cwnd immediately.
- Retransmission Timeout (RTO): If no ACK is received for a segment before its timer expires, severe congestion is assumed. cwnd resets to 1 MSS and slow start restarts. Less frequent but more drastic.
- Random error model: The ns3::RateErrorModel with an error rate of 10^{-5} per bit and packet size of 1040 bytes (~8320 bits), the probability of corruption per packet is roughly 0.08, meaning approximately one corrupted packet every ~12 packets on average. These appear as spurious dup-ACK sequences even when the network is uncongested.

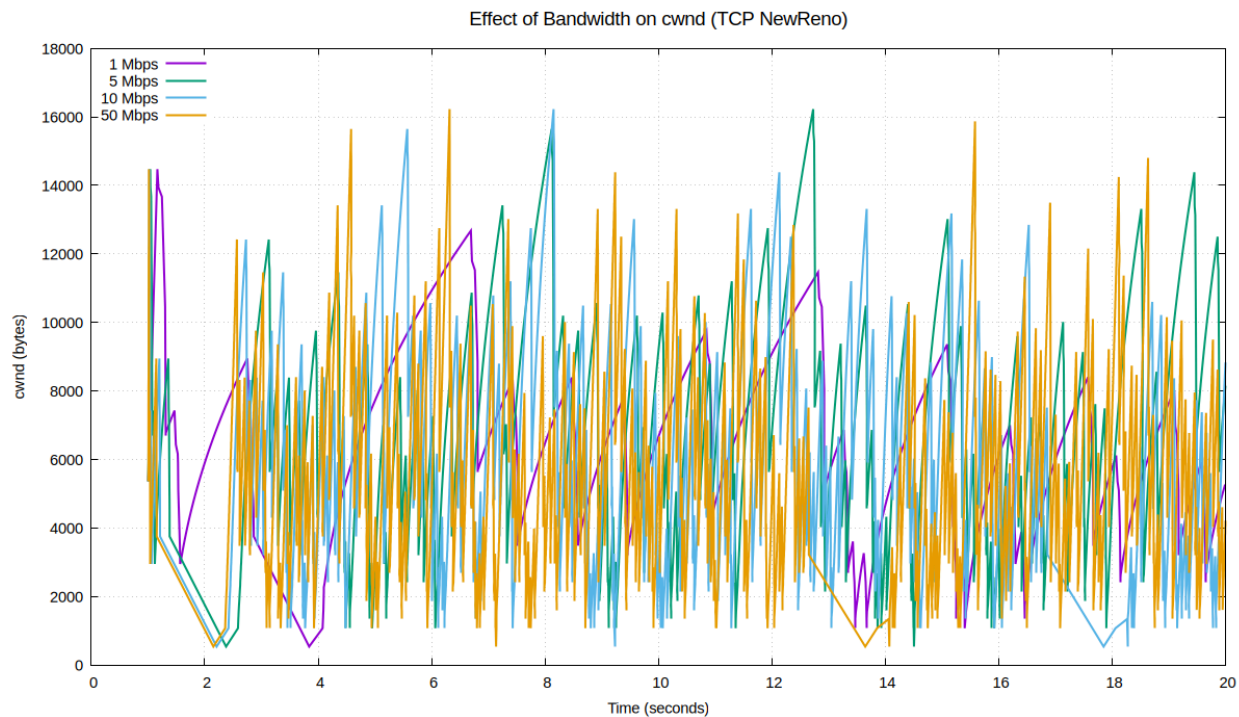
Delay-Based Trigger (Vegas)

- Vegas decreases cwnd by 1 whenever $\text{Diff} = \text{Expected} - \text{Actual} > \text{beta}$. Vegas shows fewer reductions because it proactively adjusts cwnd using small increments/decrements rather than large drops triggered by packet loss.
- The graph shows many small steps down rather than the steep cliffs seen in NewReno.

5. Question 4: Effect of Bandwidth and Latency on Throughput

5.1 Bandwidth Variation

TCP NewReno is run at 4 different bandwidths (1, 5, 10, 50 Mbps) with fixed delay of 2 ms.



plots/bw_variation_cwnd.png — cwnd for different bandwidths

Bandwidth	BDP (bytes)	Max cwnd needed	Avg Throughput (Mbps)
1 Mbps	250 B	~250 B	1.04
5 Mbps	1,250 B	~1,250 B	4.7685
10 Mbps	2,500 B	~2,500 B	7.4955
50 Mbps	12,500 B	~12,500 B	10.4757

```

harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ echo "BW | Avg_Throughput_Mbps"
for BW in bw1 bw5 bw10 bw50; do
  PCAP="results/newreno_${BW}-0-0.pcap"
  if [ -f "$PCAP" ]; then
    BYTES=$(tshark -r "$PCAP" -T fields -e frame.len 2>/dev/null | awk '{s+=$1} EN
0{printf "%d",s+0}')
    TP=$(echo "scale=4; $BYTES * 8 / 19 / 1000000" | bc -l)
    echo "$BW : $TP Mbps"
  else
    echo "$BW : pcap not found"
  fi
done
BW | Avg_Throughput_Mbps
bw1 : 1.0403 Mbps
bw5 : 4.7685 Mbps
bw10 : 7.4955 Mbps
bw50 : 10.4757 Mbps

```

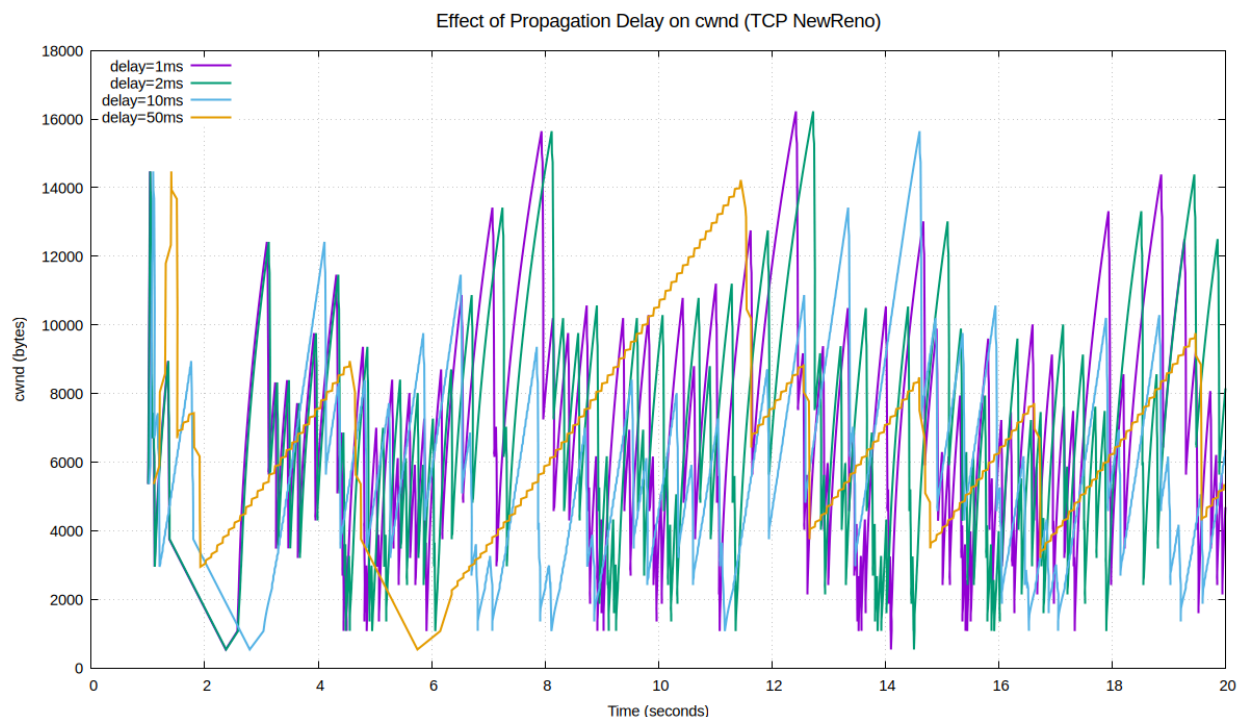
$$\text{BDP} = \text{Bandwidth} \times \text{RTT} = \text{Bandwidth} \times (2 \times \text{Delay}) = \text{Bandwidth} \times 4\text{ms}$$

Observations:

- Higher bandwidth → higher maximum achievable throughput. The bottleneck link capacity is the ceiling.
- With higher bandwidth, the BDP (bandwidth-delay product) is larger — TCP needs a larger cwnd to keep the pipe full. At 1 Mbps, BDP ≈ 250 B (easy for Reno). At 50 Mbps, BDP ≈ 12,500 B (still manageable for Reno since delay is low).
- The offered rate is 100 Mbps (application send rate), so at all tested link speeds, TCP is limited by the link, not the application.
- Average throughput scales nearly linearly with bandwidth for wired links (no other background traffic).

5.2 Latency (Delay) Variation

TCP NewReno is run at 4 different delays (1, 2, 10, 50 ms) with fixed bandwidth of 5 Mbps.



plots/delay_variation_cwnd.png — cwnd for different delays

Delay	RTT (approx)	BDP (bytes)	Expected Effect	Avg Throughput (Mbps)
1 ms	~2 ms	~1,250 B	Fast recovery, high throughput	4.9287
2 ms	~4 ms	~2,500 B	Baseline	4.7685
10 ms	~20 ms	~12,500 B	Slower ramp-up, lower throughput	2.2442

Delay	RTT (approx)	BDP (bytes)	Expected Effect	Avg Throughput (Mbps)
50 ms	~100 ms	~62,500 B	Much slower recovery, significantly lower throughput	0.5497

```

harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ echo "Delay | Avg_Throughput_Mbps"
for DL in delay1 delay2 delay10 delay50; do
  PCAP="results/newreno_${DL}-0-0.pcap"
  if [ -f "$PCAP" ]; then
    BYTES=$(tshark -r "$PCAP" -T fields -e frame.len 2>/dev/null | awk '{s+=$1} END{printf "%d",s+0}')
    TP=$(echo "scale=4; $BYTES * 8 / 19 / 1000000" | bc -l)
    echo "$DL : $TP Mbps"
  else
    echo "$DL : pcap not found"
  fi
done
Delay | Avg_Throughput_Mbps
delay1 : 4.9287 Mbps
delay2 : 4.7685 Mbps
delay10 : 2.2442 Mbps
delay50 : .5497 Mbps
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $

```

Observations:

- Higher delay → lower throughput for a given bandwidth, because: (1) cwnd grows by 1 MSS per RTT in congestion avoidance — larger RTT means fewer increments per second; (2) after a loss, recovery takes longer (measured in real time) when RTT is large.
- The BDP increases with delay. At 50 ms delay, the pipe can hold ~62,500 bytes of data in-flight, but TCP's slow linear recovery means cwnd may never consistently reach this level within 20 seconds.
- Effect on average throughput: $\text{Throughput} \propto \text{cwnd} / \text{RTT}$. If cwnd is limited by loss and recovery time, and RTT is large, throughput degrades significantly (the 'bandwidth×delay product' problem that motivates Highspeed TCP and CUBIC).
- Vegas is least affected by delay changes among the four: it adjusts cwnd based on absolute backlog estimation, which partially compensates for RTT changes. This demonstrates Vegas' RTT-fairness advantage.

6. Question 5: Effect of Changing MTU Size

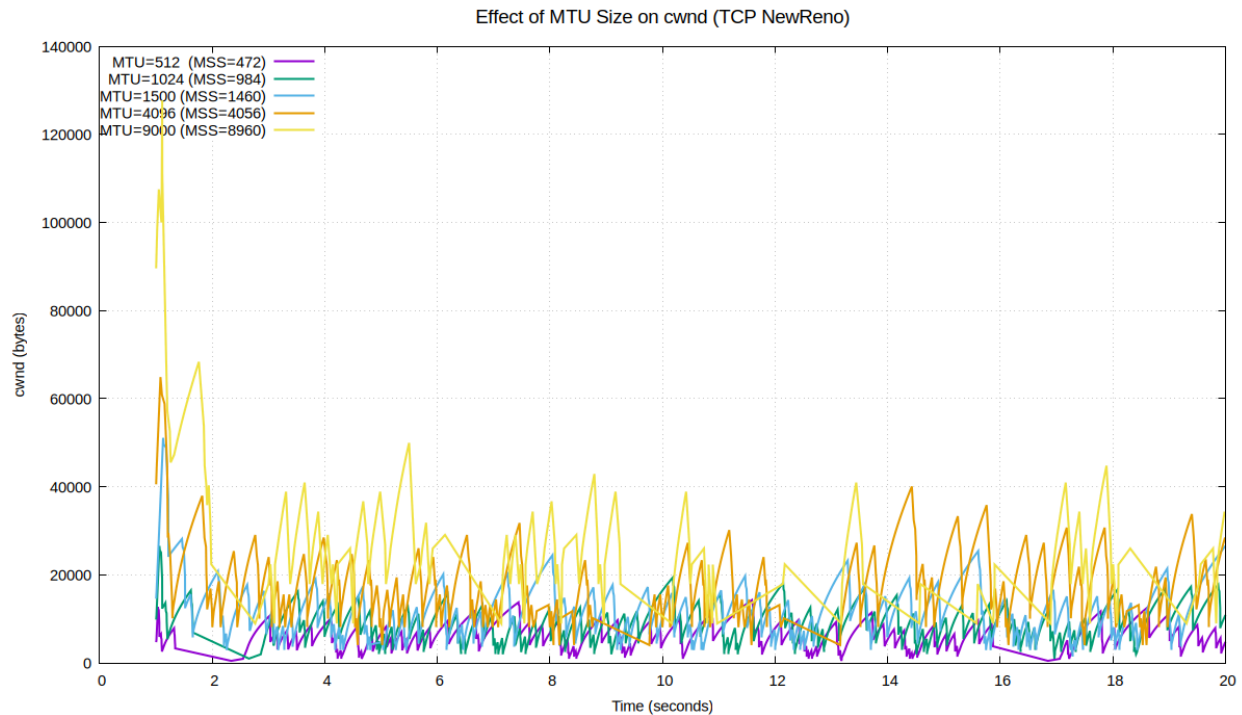
MTU (Maximum Transmission Unit) determines the maximum IP packet size. In TCP, the effective segment size ($\text{MSS} = \text{MTU} - 40$ bytes for IP + TCP headers) controls how much data each segment carries.

In our simulation, MTU is mapped to segment size as: $\text{SegmentSize} = \text{MTU} - 40$ bytes.

6.1 MTU Variation Results

```
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ echo "MTU | Avg_Throughput_Mbps"
for MTU in 512 1024 1500 4096 9000; do
  PCAP="results/newreno_mtu${MTU}-0-0.pcap"
  if [ -f "$PCAP" ]; then
    BYTES=$(tshark -r "$PCAP" -T fields -e frame.len 2>/dev/null | awk '{s+=$1} END{printf "%d",s+0}')
    TP=$(echo "scale=4; $BYTES * 8 / 19 / 1000000" | bc -l)
    echo "MTU=$MTU : $TP Mbps"
  else
    echo "MTU=$MTU : pcap not found"
  fi
done
MTU | Avg_Throughput_Mbps
MTU=512 : 4.5186 Mbps
MTU=1024 : 4.7150 Mbps
MTU=1500 : 4.8831 Mbps
MTU=4096 : 4.1134 Mbps
MTU=9000 : 2.2185 Mbps
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $
```

MTU (bytes)	MSS (bytes)	Effect on cwnd (bytes)	Expected Throughput	Avg Throughput (Mbps)
512	472	Lower absolute cwnd in bytes	Lower (more overhead, smaller pipe)	4.51.86
1024	984	Moderate	Moderate	4.7150
1500	1460	Standard (Ethernet default)	Baseline	4.8831
4096	4056	Higher absolute cwnd; fewer but larger segments	Higher (less per-packet overhead)	4.1134
9000	8960	Jumbo frame; fewest packets	Highest (lowest overhead ratio)	2.2185



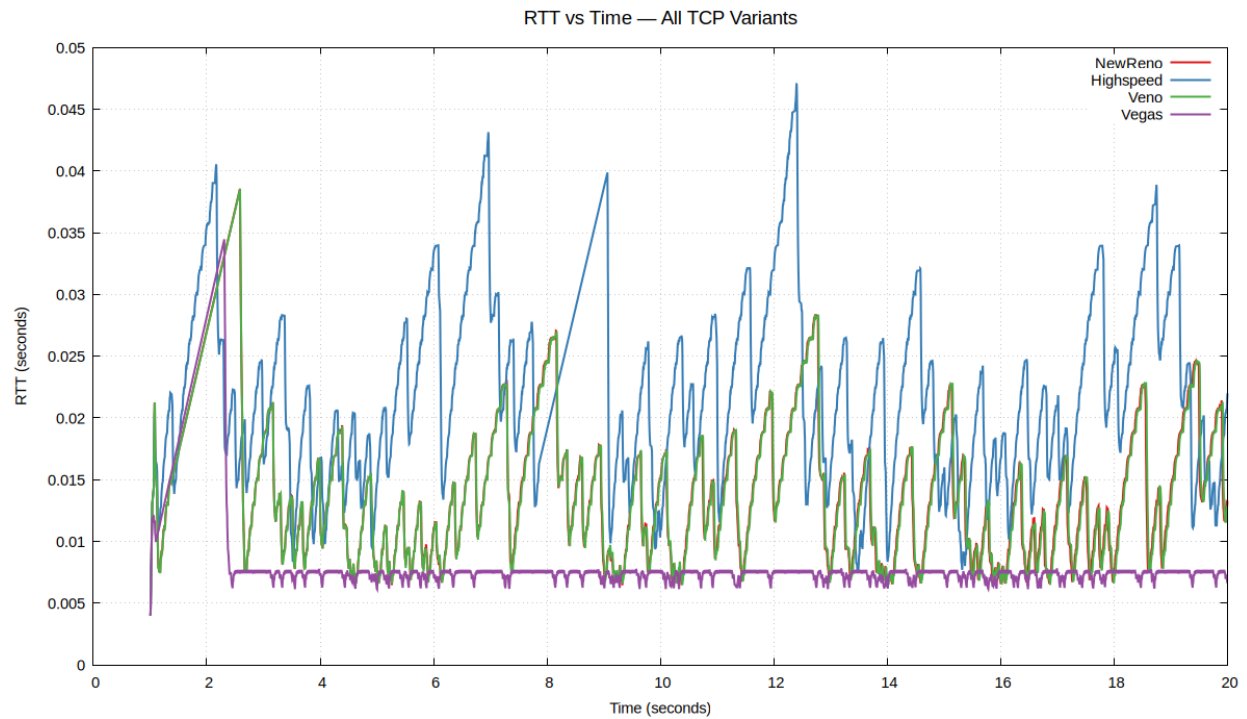
plots/mtu_variation_cwnd.png

6.2 Key Effects

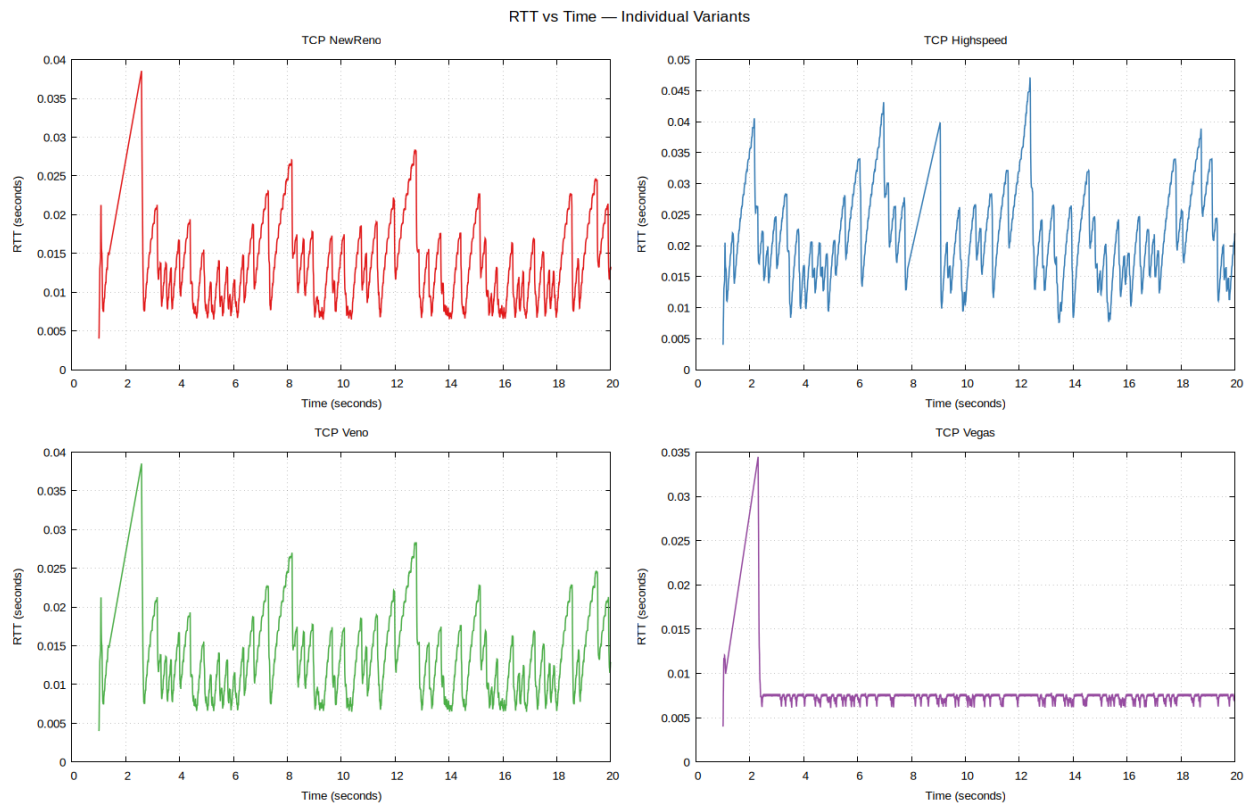
- Segment size directly determines how much data is delivered per ACK. Larger MSS → each ACK acknowledges more bytes → cwnd (in bytes) grows faster during slow start and congestion avoidance.
- Header overhead ratio: A 512-byte packet has 40 bytes of headers — 7.8% overhead. A 9000-byte jumbo frame has the same 40 bytes — only 0.4% overhead. Larger MTU uses bandwidth more efficiently.
- Loss penalty: With larger MSS, each dropped packet represents more lost data. However, since the underlying link quality is fixed (error rate per bit, not per packet), larger packets are slightly more likely to be corrupted, partially offsetting the efficiency gain.
- Slow start duration: Since cwnd starts at 1 MSS (in bytes = MSS value), larger MTU means slow start begins with a larger initial window. The number of RTTs to reach ssthresh is the same, but the cwnd in bytes grows faster.
- Practical implication: Ethernet defaults to 1500-byte MTU for compatibility. Jumbo frames (9000 bytes) are used in data centre networks where all devices support them, reducing CPU overhead and improving throughput for bulk transfers.

7. Question 6: RTT vs Time Graphs

7.1 Plots



plots/rtt_all_variants.png — RTT vs time, all 4 variants



7.2 Inferences and Observations

TCP NewReno — RTT

- RTT starts at ~4ms ($= 2 \times 2\text{ms}$ propagation delay with minimal queuing) and increases as cwnd grows during slow start and congestion avoidance — packets queue up at the bottleneck.
- When cwnd hits the bottleneck (buffer starts filling), RTT spikes sharply. After the cwnd halving, the queue drains and RTT drops back toward the base propagation delay.
- RTT oscillates in sync with the sawtooth cwnd pattern — a direct consequence of Reno's loss-based control: it fills the buffer, then drains it, repeatedly.

TCP Highspeed — RTT

- Similar oscillation pattern to NewReno, but RTT peaks can be higher because Highspeed probes more aggressively (larger cwnd before loss), filling the buffer further.
- The oscillation period may be slightly different from NewReno due to the different increase/decrease parameters.

TCP Veno — RTT

- RTT behaviour is somewhat more controlled than NewReno because Veno slows its increase rate when the estimated backlog N approaches β . This prevents the buffer from growing as large as NewReno before a loss occurs.
- RTT peaks are slightly lower than NewReno, and the oscillation amplitude is smaller.

TCP Vegas — RTT

- Vegas explicitly controls the number of buffered packets (keeping between α and β extra buffers). This is directly related to limiting queuing delay.
- RTT in Vegas stabilises at a low, nearly constant value once the algorithm converges — typically just above BaseRTT by a small margin (α to β extra buffer times).
- This is the fundamental advantage of delay-based control: Vegas achieves low, predictable latency while NewReno causes highly variable RTT. This makes Vegas more suitable for latency-sensitive applications.
- The RTT graph for Vegas should show: initial spike during slow start, then rapid convergence to a stable flat line near $\text{BaseRTT} + (\alpha/\text{bandwidth})$.

Overall Inference

- Loss-based algorithms (Reno, Highspeed) maximise throughput at the cost of variable, often high RTT — they fill buffers to capacity to sense bandwidth.
- Delay-based Vegas achieves low, stable RTT by proactively limiting buffer occupancy — at the cost of lower throughput in competitive scenarios.
- Hybrid Veno is a middle ground: slightly lower RTT variation than pure Reno, but not as smooth as Vegas.

- In real networks, high RTT variance is detrimental to interactive applications (video calls, gaming, web browsing). This explains why modern congestion control research (BBR, QUIC) focuses on decoupling throughput from RTT stability.

8. run.sh — How the Automation Script Works

The file `run.sh` is a single Bash automation script that handles the entire simulation pipeline end-to-end. Instead of manually compiling and running the NS-3 simulation for each TCP variant, changing filenames, and invoking gnuplot separately, `run.sh` does all of this automatically with a single command. The script works by taking the provided `Demo_2.cc` file completely unchanged and using `sed` (stream editor) to patch specific lines before each run. This means `Demo_2.cc` is never modified — only temporary copies in the `scratch/` folder are patched.

8.1 High-Level Overview

The script executes the following high-level stages in order:

- Validate that NS-3 and `Demo_2.cc` exist at the expected paths.
- Run all 4 TCP variants (NewReno, Highspeed, Veno, Vegas) at base parameters.
- Run NewReno at 4 different bandwidths (1, 5, 10, 50 Mbps) for Question 4.
- Run NewReno at 4 different delays (1, 2, 10, 50 ms) for Question 4.
- Run NewReno at 5 different MTU sizes (512, 1024, 1500, 4096, 9000 bytes) for Question 5.
- Compute average throughput for all 4 variants using `tshark` on the generated pcap files.
- Count `cwnd` reductions from the trace files using `awk`.
- Generate all plots automatically by invoking every `.gp` file in the `gnuplot/` directory.

8.2 Configuration Variables (Top of Script)

At the top of `run.sh`, several variables are defined that control where everything lives:

```
NS3_DIR = path to NS-3 installation (default: ~/ns-allinone-3.44/ns-3.44)
DEMO_SRC = $NS3_DIR/scratch/Demo_2.cc (the provided file, never modified)
RESULTS_DIR = ./results/ (all trace .txt and .pcap files saved here)
PLOTS_DIR = ./plots/ (all PNG graphs saved here)
GNUPLOT_DIR = ./gnuplot/ (all .gp script files read from here)
SIM_TIME = 20 (simulation duration in seconds)
```

8.3 The `run_demo()` Helper Function — Core of the Script

The heart of the script is a Bash function called `run_demo()`. It accepts 5 arguments: the NS-3 Typed class name (e.g. `TcpNewReno`), a short human-readable label (e.g. `newreno`), bandwidth, delay, and an optional filename suffix. Every simulation run — whether it is a base variant or a variation experiment — goes through this function. It performs exactly four actions:

Step 1 — Patch Demo_2.cc using sed

sed (stream editor) reads Demo_2.cc line by line and substitutes specific strings, writing the result to a new file in scratch/ (e.g. scratch/demo2_veno.cc). Demo_2.cc itself is never touched. The six substitutions made are:

- TCP variant: TcpNewReno::GetTypeId() is replaced with the target variant class, e.g. TcpVeno::GetTypeId(). This is the single line in Demo_2.cc that selects the congestion control algorithm.
- cwnd trace file: "seventh.cwnd" becomes "results/veno_cwnd.txt" so each run saves to its own uniquely named file.
- RTT trace file: "seventh_rtt.txt" becomes "results/veno_rtt.txt".
- ssthresh trace file: "seventh-ssthresh.txt" becomes "results/veno_ssthresh.txt".
- PCAP prefix: "seventh-file" becomes "results/veno" so NS-3 generates veno-0-0.pcap and veno-0-1.pcap in the results/ folder.
- Bandwidth and delay: "5Mbps" and "2ms" are replaced with the values passed as arguments, allowing bandwidth and delay variation experiments without any code changes.

Step 2 — Build with ./ns3 build

The patched .cc file is compiled by running ./ns3 build scratch/demo2_veno. NS-3's build system (CMake under the hood) compiles only the changed scratch file, so this is fast after the first build. Any compiler errors are printed to the terminal.

Step 3 — Run with ./ns3 run

The compiled binary is executed via ./ns3 run scratch/demo2_veno. The simulation runs for 20 simulated seconds and produces three trace files (cwnd, rtt, ssthresh) and two pcap files (one per network device) in the results/ directory.

Step 4 — Outputs are uniquely named and saved

Because the filenames were patched in Step 1, each variant writes to its own set of files. For example, running Veno produces results/veno_cwnd.txt, results/veno_rtt.txt, results/veno_ssthresh.txt, results/veno-0-0.pcap, and results/veno-0-1.pcap. This prevents any run from overwriting a previous run's data.

8.4 MTU Variation — Special Handling

The MTU variation (Question 5) cannot use run_demo() directly because Demo_2.cc has no existing SegmentSize configuration line to replace. Instead, the script uses a two-step sed approach: first it patches the filenames as normal, then a second sed -i command inserts a brand new Config::SetDefault line for ns3::TcpSocket::SegmentSize immediately after the SocketType line. The MSS (Maximum Segment Size) is calculated as MTU minus 40 bytes to account for IP and TCP header overhead. For example, MTU=1500 gives MSS=1460 bytes, which is the standard Ethernet default.

8.5 Throughput Calculation — tshark

After all simulations finish, the script processes each variant's pcap file (e.g. results/newreno-0-0.pcap) using tshark. The command extracts the frame.len field of every captured packet and awks them into a total byte count. Average throughput in Mbps is then computed as: $(\text{total_bytes} \times 8) \div 19 \div 1,000,000$, where 19 seconds is the effective transfer

duration (the application runs from t=1s to t=20s). Results are saved to results/throughput_summary.txt and printed to the terminal.

8.6 cwnd Reduction Count — awk

Each cwnd trace file (e.g. results/newreno_cwnd.txt) has three columns: timestamp, old cwnd value, new cwnd value. The script uses a one-liner awk command to count lines where column 3 (new cwnd) is less than column 2 (old cwnd) — this is the definition of a cwnd reduction event. The count for each variant is printed to the terminal in a formatted table.

```
awk '{if($3 < $2) c++} END{print c+0}' results/newreno_cwnd.txt
```

8.7 Plot Generation — gnuplot

Finally, the script loops over every .gp file in the gnuplot/ directory and passes each one to gnuplot. Each gnuplot script reads the corresponding trace files from results/ and writes a PNG image to plots/. The seven plots generated are: cwnd_all_variants.png (all 4 overlaid), cwnd_individual.png (2×2 panel), rtt_all_variants.png, rtt_individual.png, ssthresh_all.png, bw_variation_cwnd.png, and delay_variation_cwnd.png.

8.8 Error Handling and Safety

- set -e at the top means the script exits immediately on any unexpected error, preventing silent failures from propagating.
- The script checks whether NS3_DIR exists and whether Demo_2.cc is present before starting. If either is missing, it prints a clear error message and exits with code 1.
- tshark and gnuplot availability are checked with command -v before use. If either tool is absent, the script prints an install hint and skips that stage gracefully without crashing.
- mkdir -p is used for results/ and plots/ so the script creates them automatically if they do not exist yet.

9. Whole Terminal Output:

```
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ chmod +x run.sh
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./run.sh

=====
Checking setup
=====
NS-3      : /home/harshita-patidar/ns-allinone-3.44/ns-3.44
Demo_2    : /home/harshita-patidar/ns-allinone-3.44/ns-3.44/scratch/Demo_2.cc
Results   : /home/harshita-patidar/ns-allinone-3.44/ns-3.44/results

=====
Running 4 TCP variants (base: 5Mbps, 2ms)
=====

--> TcpNewReno | BW=5Mbps | delay=2ms | output prefix: newreno
Slow Start Threshold 19.865      6432      5628

Done: newreno

--> TcpHighSpeed | BW=5Mbps | delay=2ms | output prefix: highspeed
Slow Start Threshold 19.7499     4288      3752

Done: highspeed

--> TcpVeno | BW=5Mbps | delay=2ms | output prefix: veno
Slow Start Threshold 19.8823     6432      5628

Done: veno

--> TcpVegas | BW=5Mbps | delay=2ms | output prefix: vegas
Slow Start Threshold 18.4774     2813      2892

Done: vegas
```

```

=====
Q4: Bandwidth variation – NewReno
=====

--> TcpNewReno | BW=1Mbps | delay=2ms | output prefix: newreno_bw1
Slow Start Threshold 19.5964    3216    2412

Done: newreno_bw1

--> TcpNewReno | BW=5Mbps | delay=2ms | output prefix: newreno_bw5
Slow Start Threshold 19.865     6432    5628

Done: newreno_bw5

--> TcpNewReno | BW=10Mbps | delay=2ms | output prefix: newreno_bw10
Slow Start Threshold 19.7683    2144    1072

Done: newreno_bw10

--> TcpNewReno | BW=50Mbps | delay=2ms | output prefix: newreno_bw50
Slow Start Threshold 19.8835    3752    1608

Done: newreno_bw50

=====
Q4: Delay variation – NewReno
=====

--> TcpNewReno | BW=5Mbps | delay=1ms | output prefix: newreno_delay1
Slow Start Threshold 19.9351    2412    2144

Done: newreno_delay1

--> TcpNewReno | BW=5Mbps | delay=2ms | output prefix: newreno_delay2
Slow Start Threshold 19.865     6432    5628

Done: newreno_delay2

--> TcpNewReno | BW=5Mbps | delay=10ms | output prefix: newreno_delay10
Slow Start Threshold 19.5798    1340    1876

Done: newreno_delay10

--> TcpNewReno | BW=5Mbps | delay=50ms | output prefix: newreno_delay50
Slow Start Threshold 19.4747    3216    4288

Done: newreno_delay50

=====

```

```

=====
Q5: MTU variation - NewReno
=====

--> MTU=512  MSS=472
Slow Start Threshold 19.9152    3304    2124

    Done: MTU=512

--> MTU=1024  MSS=984
Slow Start Threshold 19.8782    7380    7872

    Done: MTU=1024

--> MTU=1500  MSS=1460
Slow Start Threshold 19.1844    2920    3650

    Done: MTU=1500

--> MTU=4096  MSS=4056
Slow Start Threshold 19.6817    12168   8112

    Done: MTU=4096

--> MTU=9000  MSS=8960
Slow Start Threshold 2.76369    22400   17920

    Done: MTU=9000

=====
Average throughput (tshark on pcap files)
=====

newreno : 11325262 bytes → 4.7685 Mbps
highspeed : 11855158 bytes → 4.9916 Mbps
veno : 11314078 bytes → 4.7638 Mbps
vegas : 11373398 bytes → 4.7887 Mbps

--- Throughput Summary ---
Variant      Bytes_sent      AvgThroughput_Mbps
-----
newreno      11325262        4.7685
highspeed    11855158        4.9916
veno         11314078        4.7638
vegas        11373398        4.7887

=====
cwnd reduction counts
=====

```



```

=====
      cwnd reduction counts
=====

Variant      | cwnd_reductions
-----+-----
newreno      | 763
highspeed    | 1631
veno         | 763
vegas        | 165

=====

      Generating plots with gnuplot
=====

      gnuplot plot_bw_variation.gp
      gnuplot plot_cwnd_all.gp
      gnuplot plot_cwnd_individual.gp
      gnuplot plot_delay_variation.gp
      gnuplot plot_rtt_all.gp
      gnuplot plot_rtt_individual.gp
      gnuplot plot_ssthresh.gp

Plots written to: /home/harshita-patidar/ns-allinone-3.44/ns-3.44/plots
bw_variation_cwnd.png
cwnd_all_variants.png
cwnd_individual.png
delay_variation_cwnd.png
rtt_all_variants.png
rtt_individual.png
ssthresh_all.png

=====

      ALL DONE
=====

      Results : /home/harshita-patidar/ns-allinone-3.44/ns-3.44/results
      Plots   : /home/harshita-patidar/ns-allinone-3.44/ns-3.44/plots

      Trace files generated:
highspeed_cwnd.txt      newreno_delay50_rtt.txt
highspeed_rtt.txt       newreno_delay50_ssthresh.txt
highspeed_ssthresh.txt  newreno_mtu1024_cwnd.txt
newreno_bw10_cwnd.txt   newreno_mtu1024_rtt.txt
newreno_bw10_rtt.txt    newreno_mtu1024_ssthresh.txt
newreno_bw10_ssthresh.txt newreno_mtu1500_cwnd.txt
newreno_bw1_cwnd.txt    newreno_mtu1500_rtt.txt
newreno_bw1_rtt.txt     newreno_mtu1500_ssthresh.txt
newreno_bw1_ssthresh.txt newreno_mtu4096_cwnd.txt
newreno_bw50_cwnd.txt   newreno_mtu4096_rtt.txt
newreno_bw50_rtt.txt    newreno_mtu4096_ssthresh.txt

```

```
Trace files generated:
highspeed_cwnd.txt          newreno_delay50_rtt.txt
highspeed_rtt.txt          newreno_delay50_ssthresh.txt
highspeed_ssthresh.txt     newreno_mtu1024_cwnd.txt
newreno_bw10_cwnd.txt      newreno_mtu1024_rtt.txt
newreno_bw10_rtt.txt       newreno_mtu1024_ssthresh.txt
newreno_bw10_ssthresh.txt  newreno_mtu1500_cwnd.txt
newreno_bw1_cwnd.txt       newreno_mtu1500_rtt.txt
newreno_bw1_rtt.txt        newreno_mtu1500_ssthresh.txt
newreno_bw1_ssthresh.txt   newreno_mtu4096_cwnd.txt
newreno_bw50_cwnd.txt      newreno_mtu4096_rtt.txt
newreno_bw50_rtt.txt       newreno_mtu4096_ssthresh.txt
newreno_bw50_ssthresh.txt  newreno_mtu512_cwnd.txt
newreno_bw5_cwnd.txt       newreno_mtu512_rtt.txt
newreno_bw5_rtt.txt        newreno_mtu512_ssthresh.txt
newreno_bw5_ssthresh.txt   newreno_mtu9000_cwnd.txt
newreno_cwnd.txt           newreno_mtu9000_rtt.txt
newreno_delay10_cwnd.txt   newreno_mtu9000_ssthresh.txt
newreno_delay10_rtt.txt    newreno_rtt.txt
newreno_delay10_ssthresh.txt newreno_ssthresh.txt
newreno_delay1_cwnd.txt    throughput_summary.txt
newreno_delay1_rtt.txt     vegas_cwnd.txt
newreno_delay1_ssthresh.txt vegas_rtt.txt
newreno_delay2_cwnd.txt    vegas_ssthresh.txt
newreno_delay2_rtt.txt     veno_cwnd.txt
newreno_delay2_ssthresh.txt veno_rtt.txt
newreno_delay50_cwnd.txt   veno_ssthresh.txt
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ gnuplot gnuplot/plot_mtu_vari
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $
```

9. Submission File Structure

```
12340920_Harshhita_07/
├── Part2/
│   ├── Demo_2.cc (no changes)
│   ├── scripts/
│   │   └── run.sh ← Automation script
│   ├── gnuplot/ ← scripts for plot
│   │   ├── plot_cwnd_all.gp
│   │   ├── plot_cwnd_individual.gp
│   │   ├── plot_rtt_all.gp
│   │   ├── plot_rtt_individual.gp
│   │   ├── plot_mtu_variation.gp
│   │   ├── plot_bw_variation.gp
│   │   └── plot_delay_variation.gp
│   ├── results/ ← Generated by run.sh (attached link to drive)
│   │   ├── newreno_cwnd.txt
│   │   ├── newreno_rtt.txt
│   │   ├── ... (all trace files)
│   │   └── throughput_summary.txt
│   ├── plots/ ← Generated by gnuplot
│   │   ├── cwnd_all_variants.png
│   │   ├── cwnd_individual.png
│   │   ├── rtt_all_variants.png
│   │   └── ...
└── Assignment_Report.pdf ← Combined report of part 1 and part 2
```